# VoteChain Private Blockchain Project - Team Roadmap

**Timeline:** 17 Sept 2025 → 13 Feb 2026
**Target Users:** ~1,000
**Roles:**
- You + Rajib: Core blockchain developers (C++ + consensus + hashes + fingerprint)
- Abhash: Frontend (React.js/Next.js)
- Sandesh: Backend (Node.js + API + database)
- Ayush: Testing & QA

---

## Phase 0: Orientation & Architecture (17–30 Sept)

### Core Devs (You + Rajib)

- Learn blockchain fundamentals in C++
- OOP design for blockchain: `Block`, `Transaction`, `Blockchain`, `Node`
- Learn fingerprint SDK basics in C++
- Learn ephemeral hash concept (permanent hash → temp election hash)
- Prepare UML & flow diagrams
- Setup development environment (VS Code/CLion, GitHub, Crypto++ / OpenSSL)

### Frontend (Abhash)

- Setup React.js/Next.js project
- Design wireframes: registration page, voting page, result dashboard

### Backend (Sandesh)

- Setup Node.js + Express, define API routes for blockchain interaction
- Design database schema for voter info + fingerprint templates

### Testing (Ayush)

- Learn unit testing basics for C++ and API
- Prepare test case templates

---

## Phase 1: Core Blockchain + Registration Module (1–31 Oct)

### Core Devs

- Implement `Transaction` class: voterID, fingerprintHash, ephemeral hash placeholder
- Implement `Block` class: index, timestamp, transactions, previousHash, hash
- Implement `Blockchain` class: addBlock(), isChainValid()
- Implement registration module: capture fingerprint + college ID + details
- Generate permanent hash (SHA256)

- Store permanent hash in backend database

### Frontend

- Registration page connected to backend
- Show confirmation of registration

### Backend

- `/registerVoter` API: store permanent hash + student info
- Dummy endpoints for temp hash testing

### Testing

- Test block creation and permanent hash generation
- Check fingerprint capture & storage

---

## Phase 2: Temporary Hash & Voting Module (1–30 Nov)

### Core Devs

- Implement temp hash generation:
  `tempHash = SHA256(permanentHash + electionName + electionDate + randomSalt)`
- Implement voting module: scan fingerprint → verify identity → generate tempFingerprintHash → cast vote → store tempHash on blockchain
- Prevent double voting by checking blockchain for existing tempHash
- Test voting with 10–50 simulated users

### Frontend

- Voting page: scan fingerprint → show student info → vote selection → submit
- Display confirmation & vote count

### Backend

- `/castVote` API → handle tempHash + vote submission
- `/verifyFingerprint` API → check scanned fingerprint against permanent hash

### Testing

- Unit tests for temp hash generation & uniqueness
- Verify double-voting prevention
- Test frontend-backend integration

---

## Phase 3: Node Network & Consensus (1–31 Dec)

### Core Devs

- Implement `Node` class: peers, broadcasting, receiving blocks

- Implement consensus: Proof-of-Authority or PBFT
- Simulate 3–5 nodes locally
- Test vote propagation and block validation across nodes
- Optimize C++ code for memory & speed

### Frontend

- Connect voting UI with node network via backend
- Real-time vote count dashboard

### Backend

- Handle node registration & API for blockchain communication
- Logging and error handling

### Testing

- Multi-node consistency testing
- Performance check for 50–100 users

---

## Phase 4: Security, Optimization & 1K User Pilot (1 Jan – 13 Feb)

### Core Devs

- Add digital signatures for votes
- Finalize ephemeral hash logic → temp hashes discarded after election
- Optimize blockchain performance for 1K users
- Document blockchain architecture & code

### Frontend

- Improve UI/UX, mobile responsiveness
- Final integration with backend & blockchain

### Backend

- Optimize API for 1K voters
- Ensure secure storage & ephemeral hash logic works

### Testing

- Load test for ~1,000 users
- Security audit: digital signatures, temp hash unlinkability
- Final bug fixes

---

## Key Learning Goals for Core Devs (You + Rajib)

1. Blockchain fundamentals from scratch in C++
2. OOP applied in real-world project
3. Fingerprint integration + biometric security

4. Permanent vs ephemeral hashes for privacy
5. Peer-to-peer network & consensus algorithm
6. Fullstack integration with frontend + backend
7. Testing & optimization for real-world scale